

```

# lab1.py

from src import linalg, distributions, plotting

# Part 1: matrix/vector operations
A = [[1, 2, 3],
      [4, 5, 6]]                # 2x3

B = [[7, 8],
      [9, 10],
      [11, 12]]                # 3x2

v1 = [1, 2, 3, 4]
v2 = [5, 6, 7, 8]

S = [[2, 1, 0],
      [1, 3, 1],
      [0, 1, 4]]                # symmetric

U = [[1, 2, 3],
      [0, 4, 5],
      [0, 0, 6]]                # upper triangular

L = [[1, 0, 0],
      [2, 3, 0],
      [4, 5, 6]]                # lower triangular

print("dot(v1, v2):", linalg.dot(v1, v2))
print("transpose(A):\n", linalg.transpose(A))
print("matmul(A, B):\n", linalg.matmul(A, B))
print("is_symmetric(S):", linalg.is_symmetric(S))
print("is_symmetric(A) [non-square]:", linalg.is_symmetric(A))
print("is_upper(U):", linalg.is_upper(U))
print("is_lower(L):", linalg.is_lower(L))

# Part 2(a): uniform samples in [a, b)
uniform_samples = distributions.uniform(a=0, b=1, n=20000)
plotting.histogram(uniform_samples, title="Uniform Distribution", bins=10)

# Part 2(b)/(c): gaussian samples with N(mu, sigma^2)
gaussian_samples = distributions.gaussian(mu=0, sigma=1, n=20000)
plotting.histogram(gaussian_samples, title="Gaussian Distribution")

```

```

# src/linalg.py

import numpy as np

# dot product
def dot(a, b):
    tot = 0
    for i in range(len(a)):
        tot += a[i] * b[i]
    return tot

# transpose mxn to nxm
def transpose(A):
    m = len(A)
    n = len(A[0])

    arr = np.zeros((n, m))
    for i in range(n):
        for j in range(m):
            arr[i, j] = A[j][i]

    return arr

# matrix multiplication
def matmul(A, B):
    #  $A (m \times n) * B (n \times k) = C (m \times k)$ 
    m = len(A)
    n = len(B)
    k = len(B[0])

    B_t = transpose(B)
    arr = np.zeros((m, k))

    for i in range(m):
        for j in range(k):
            arr[i, j] = dot(A[i], B_t[j])

    return arr

# is symmetric matrix
def is_symmetric(A):
    m = len(A)
    n = len(A[0])

    if m != n:
        return False

    for i in range(m):
        for j in range(i+1, n):
            if A[i][j] != A[j][i]:
                return False

    return True

```

```

# is upper triangular matrix
def is_upper(A):
    m = len(A)
    n = len(A[0])

    if m != n:
        return False

    # selecting the i>j region
    for i in range(1, m):
        for j in range(i):
            if A[i][j] != 0:
                return False

    return True

# is lower triangular matrix
def is_lower(A):
    m = len(A)
    n = len(A[0])

    if m != n:
        return False

    # selecting the i<j region
    for i in range(m):
        for j in range(i+1, n):
            if A[i][j] != 0:
                return False

    return True

```

```

# src/distributions.py

import numpy as np

# Uniform Distribution
def uniform(a, b, n, rng=None):
    if rng is None:
        rng = np.random.default_rng()

    samples = np.zeros(n)
    for i in range(n):
        u = rng.random()          # raw sample, uniform in [0, 1)
        samples[i] = a + (b - a) * u  # rescale u into [a, b)

    return samples

# Gaussian Distribution (polar / Marsaglia method)
def gaussian(mu, sigma, n, rng=None):
    if rng is None:
        rng = np.random.default_rng()

    samples = np.zeros(n)
    i = 0
    while i < n:
        u1 = uniform(-1, 1, 1, rng)[0]
        u2 = uniform(-1, 1, 1, rng)[0]
        s = u1 ** 2 + u2 ** 2

        if not (0 < s < 1):
            continue          # rejected -- go throw another dart

        k = np.sqrt(-2 * np.log(s) / s)
        x = u1 * k
        y = u2 * k

        samples[i] = mu + sigma * x
        i += 1
        if i < n:
            samples[i] = mu + sigma * y
            i += 1

    return samples

```

```
# src/plotting.py

import matplotlib.pyplot as plt

def histogram(data, title="", bins=30, xlabel="value", ylabel="frequency"):
    plt.figure()
    plt.hist(data, bins=bins)
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.show()
```